



United International University (UIU)
Department of Computer Science and Engineering (CSE)
Course: VLSI Design (CSE 4327)

Experiment 3: Simulation of Multiplexers using Verilog

Objectives

1. To become familiar with the Cadence Incisive Enterprise Simulator (IES).
2. To write structural and behavioral Verilog models of 2-to-1, 4-to-1, and 8-to-1 multiplexers.
3. To write self-checking test bench (stimulus) modules and verify each design using the SimVision waveform viewer.
4. To practice compiling, elaborating, and simulating a Verilog design using ncvlog, ncelab, and the simulator within NCLaunch.

Introduction

A multiplexer (MUX) is a combinational circuit that selects one of several input signals and forwards it to a single output line, based on the value of one or more selector (select) lines. Multiplexers are fundamental building blocks in digital systems, used in data routing, bus switching, and as general-purpose logic elements. In this experiment, Verilog HDL is used to describe and simulate 2-to-1, 4-to-1, and 8-to-1 multiplexers, using the Cadence Incisive Enterprise Simulator (IES) - invoked through the NCLaunch graphical front end - for compilation, elaboration, and simulation.

Part 1: Setting Up the Design Files

1. Create a lab directory and a simulation sub-directory under your work area:

```
mkdir lab3
cd lab3
mkdir simulation
cd simulation
```

2. Use an editor (vi, nedit, gedit, etc.) to write the Verilog code for a 2-to-1 multiplexer (MUX):

```
vi mux21.v
```

```
`timescale 1ns/1ps
module mux21 (
    input  i0,
    input  i1,
    input  s,
    output out
);
    assign out = (s) ? i1 : i0;
endmodule
```

Listing 1: mux21.v - structural/dataflow description of a 2-to-1 multiplexer.

3. Use an editor to write a test bench Verilog code for your 2-to-1 MUX:**vi mux21_test.v**

```
`timescale 1ns/1ps

module mux21_test;

    reg i0;
    reg i1;
    reg s;

    wire out;

    mux21 uut (
        .i0(i0),
        .i1(i1),
        .s(s),
        .out(out)
    );

    initial begin

        $monitor("Time = %0t | i0 = %b | i1 = %b | s = %b | out = %b",
            $time, i0, i1, s, out);

        i0 = 0;
        i1 = 1;
        s = 0;
        #10;

        i0 = 1;
        i1 = 0;
        s = 0;
        #10;

        i0 = 0;
        i1 = 1;
        s = 1;
        #10;

        i0 = 1;
        i1 = 0;
        s = 1;
        #10;

        i0 = 0;
        i1 = 0;
        s = 0;
        #10;

        i0 = 1;
        i1 = 1;
        s = 1;
        #10;

        $finish;
    end

endmodule
```

Listing 2: mux21_test.v - test bench that applies stimulus to mux21 and monitors its output.

4. We will use the Incisive Enterprise Simulator (IES) to verify that the MUX is working properly. Invoke the tool by typing 'nclaunch -new' in the terminal:

```
nclaunch -new
```

Part 2: Launching NCLaunch and Creating the Work Library

5. The NCLaunch window will appear. Click on 'Multiple Step'.

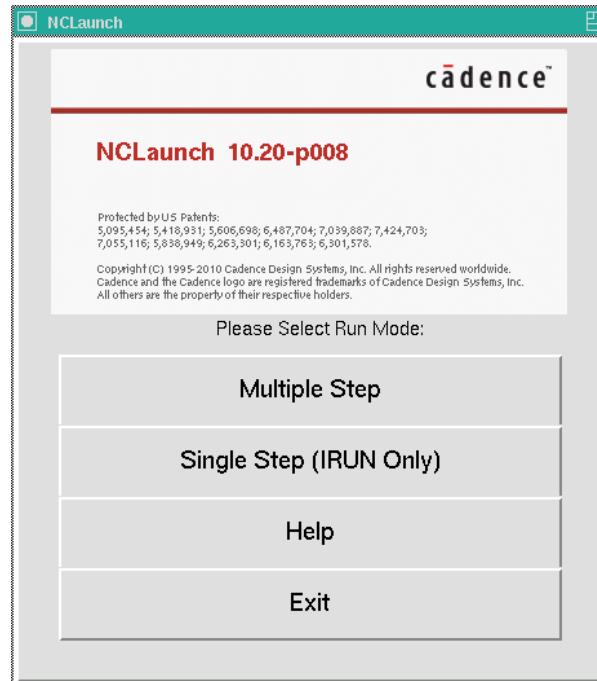


Figure 1: NCLaunch run-mode selection window.

6. The 'nclaunch: Open Design Directory' window will appear. Click on 'Create cds.lib File...'.

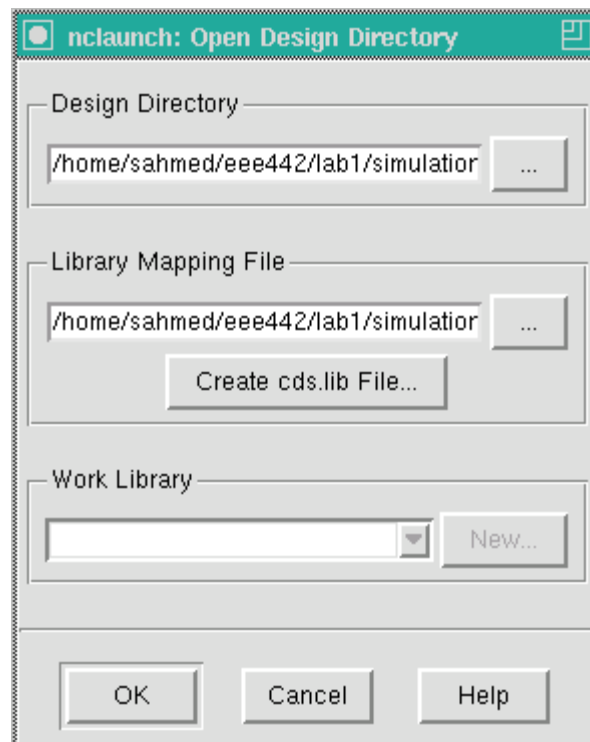


Figure 2: nclaunch Open Design Directory window, before the cds.lib file is created.

7. The 'Create a cds.lib file' window will appear. Click on 'Save'.

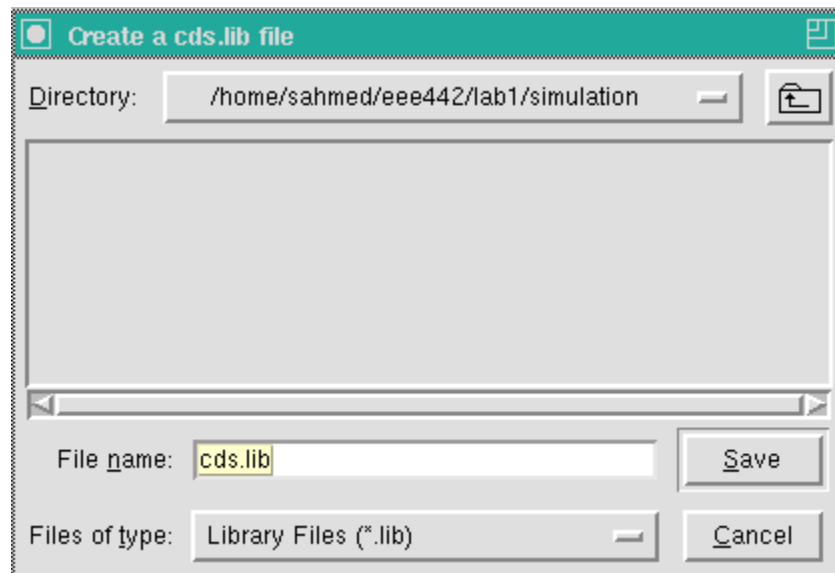


Figure 3: Create a cds.lib file window.

8. The 'New cds.lib File' window will appear. Select the third option, since we are using Verilog. Click on 'OK'.

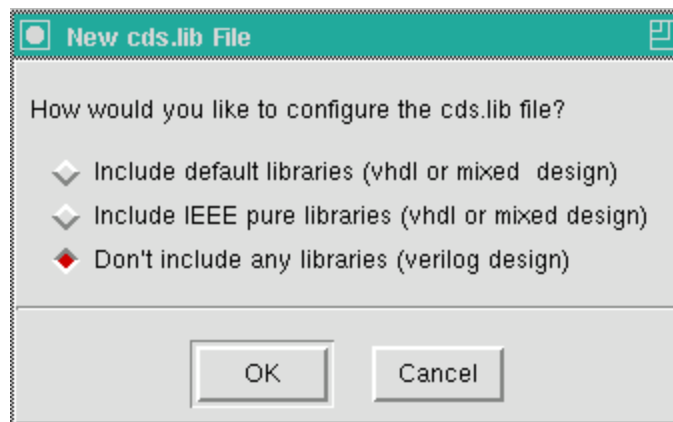


Figure 4: New cds.lib File window, with 'Don't include any libraries (Verilog design)' selected.

9. Click 'OK' in the 'nclaunch: Open Design Directory' window.

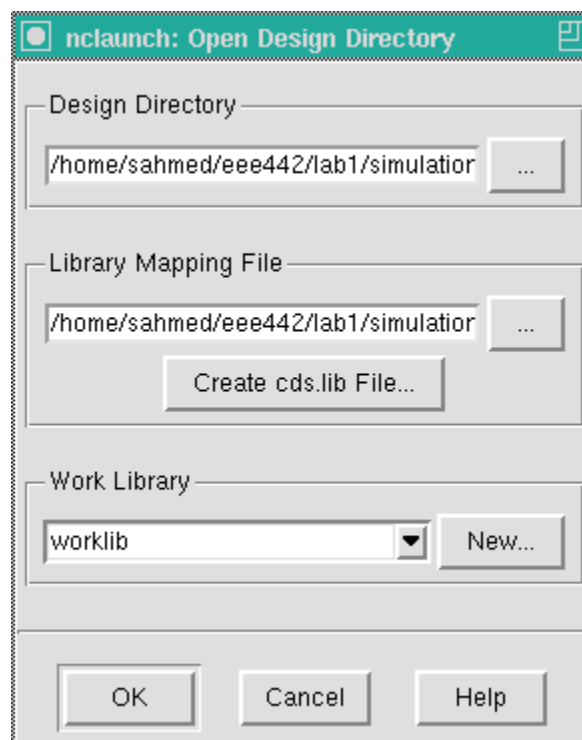


Figure 5: Open Design Directory window with the worklib now populated.

Part 3: Compiling the Design

10. In the NCLaunch window, we will see the design (mux21.v) and the test bench (mux21_test.v) that we created in the simulation directory.

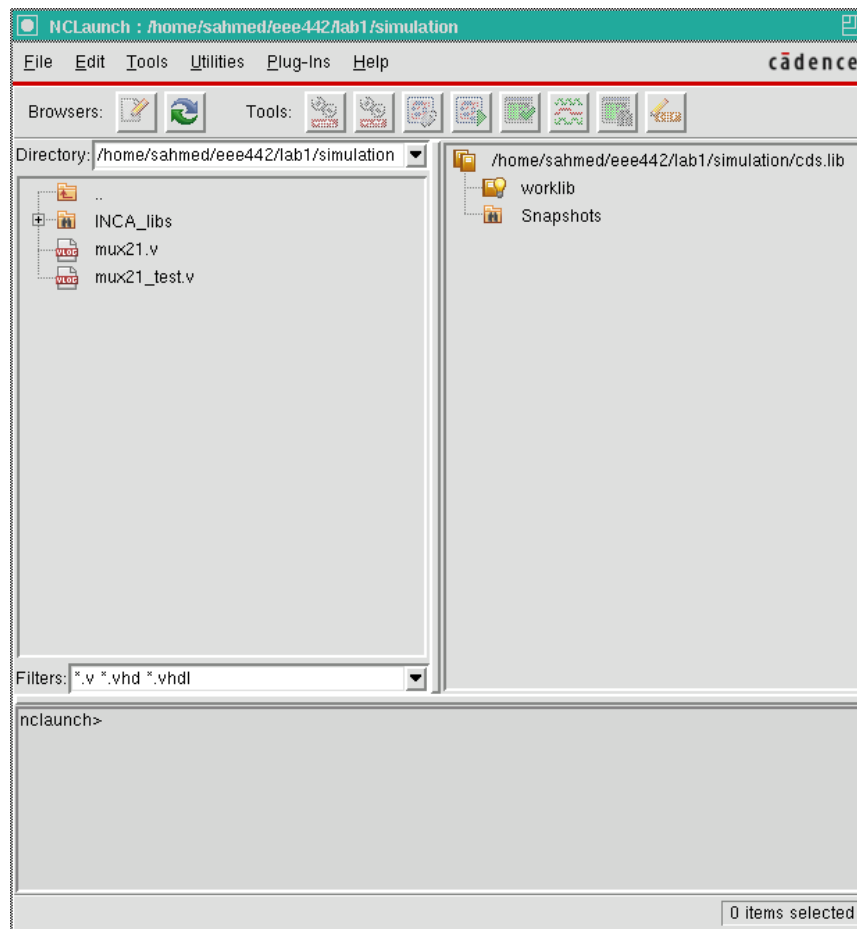


Figure 6: NCLaunch window showing mux21.v and mux21_test.v in the simulation directory.

11. Now we will COMPILE the code to check its syntax and semantics. Select both the design (mux21.v) and the test bench (mux21_test.v), then click on nclog for Verilog designs.

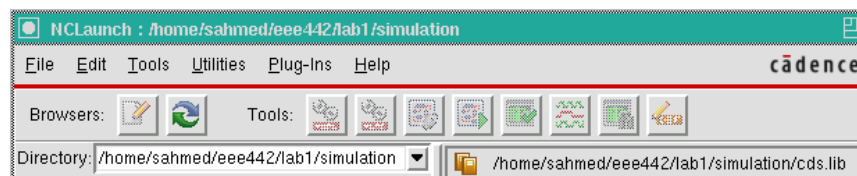


Figure 7: NCLaunch toolbar, with the nclog (compile) icon highlighted.

12. Any error in the code will be reported in the 'Command Entry Window' at the bottom of NCLaunch. Clean all the errors in the design and/or the test bench before continuing.

Part 4: Elaboration - Building the Design Hierarchy

Elaboration constructs the design hierarchy and connects the signals between the design and the test bench.

13. Open the 'worklib' directory on the right side of the window and select the **test bench**, since it is the top module (it instantiates and contains the design module). Click on the 'launch elaborator' (ncelab) icon.



Figure 8: NCLaunch toolbar, with the ncelab (elaborate) icon highlighted.

Part 5: Simulation - Running and Viewing the Waveforms

Simulation executes the elaborated design against the applied stimulus.

14. Open the 'Snapshots' directory on the right side of the window and select the worklib snapshot of the test bench. Click on 'launch simulator'.



Figure 9: NCLaunch toolbar, with the launch simulator icon highlighted.

15. A 'Console - SimVision' window will appear. This is used to perform simulation in command mode. Minimize this window, since we will be using GUI mode for simulation.
16. A 'Design Browser - SimVision' window also appears. There are many options in the Design Browser to analyze the design and for debugging. Select the test bench module on the left side of the window, then click on 'waveform'.

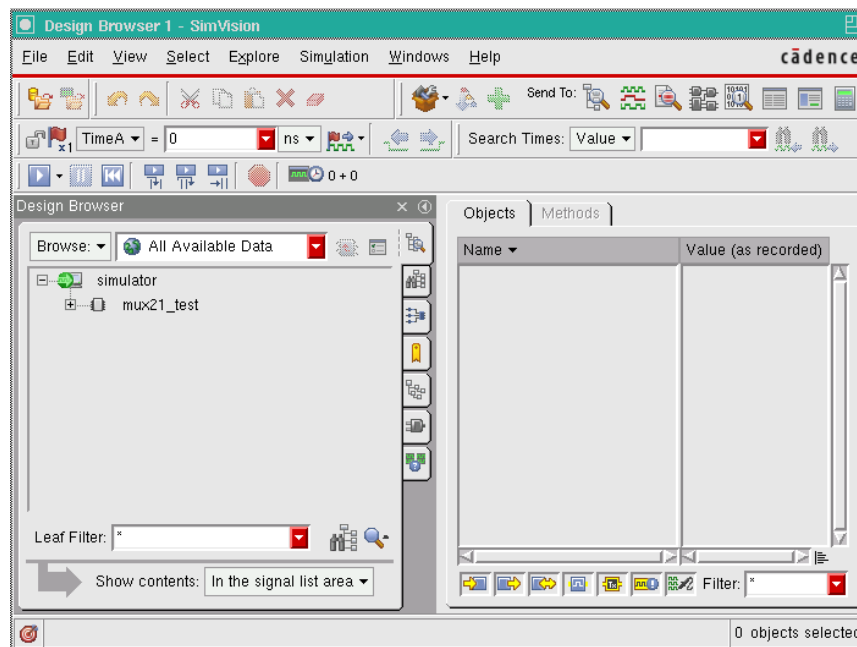


Figure 10: Design Browser - SimVision window, with the mux21_test module selected.

17. A 'Waveform - SimVision' window will appear. In the waveform window we will see the ports of the design. Click on 'run simulation' to start the simulation. Use the 'pause' button to interrupt or stop the simulation. Use options such as zoom in and zoom out to analyze the plot; the Waveform window offers many further options for analyzing the design and debugging.

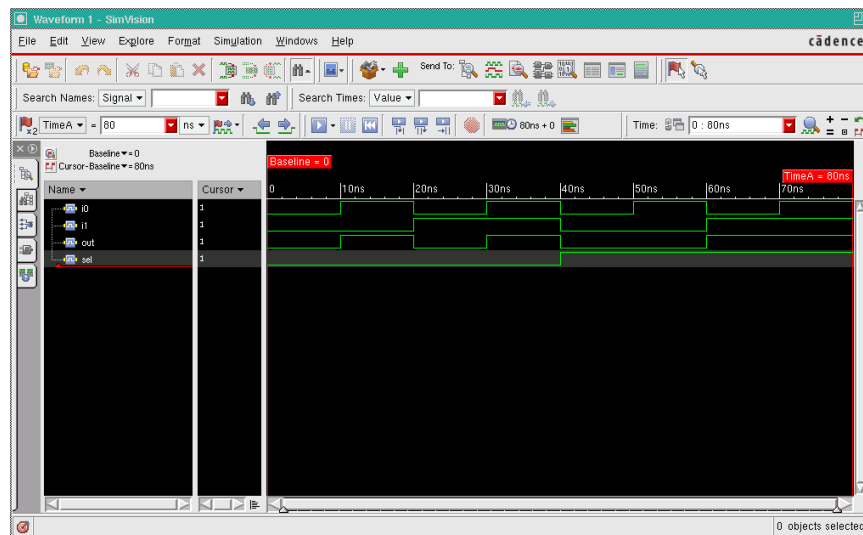


Figure 11: Waveform - SimVision window, showing the i0, i1, out and sel signals of mux21.

18. After verifying the design, close the tools.

Part 6: Designing and Verifying a 4-to-1 Multiplexer

19. Write Verilog code for a 4-to-1 MUX (mux41.v), using the 2-to-1 MUX you have just tested as a building block.
20. Write the test bench (stimulus module) for the above 4-to-1 MUX (mux41_test.v). The test bench code must be written according to the following specification:

Signal	Description
Inputs	i0, i1, i2, i3
Selector inputs	s0, s1
out = i0	for s0 = 0 and s1 = 0
out = i1	for s0 = 1 and s1 = 0
out = i2	for s0 = 0 and s1 = 1
out = i3	for s0 = 1 and s1 = 1

Choose the input values (clock-style periodic waveforms) for the stimulus as given below:

Signal	Low period	High period
i0	05	05
i1	10	10
i2	20	20
i3	40	40
s0	80	80
s1	160	160

21. Use the Incisive Enterprise Simulator (IES) to verify that the 4-to-1 MUX is working properly. Invoke the tool by typing 'nclaunch' without the '-new' option, since we will reuse the previously created worklib:

nclaunch

22. Follow the same compile, elaborate, and simulate steps described in Parts 3 to 5 above to test the 4-to-1 MUX.

Part 7: Designing and Verifying an 8-to-1 Multiplexer

23. Write a behavioral model of an 8-to-1 MUX. The design must use vector notation for its inputs and selector lines. Provide appropriate stimulus in the test bench to run the simulation.
24. Use the Incisive Enterprise Simulator (IES) to verify that the 8-to-1 MUX is working properly, following the same steps described before.

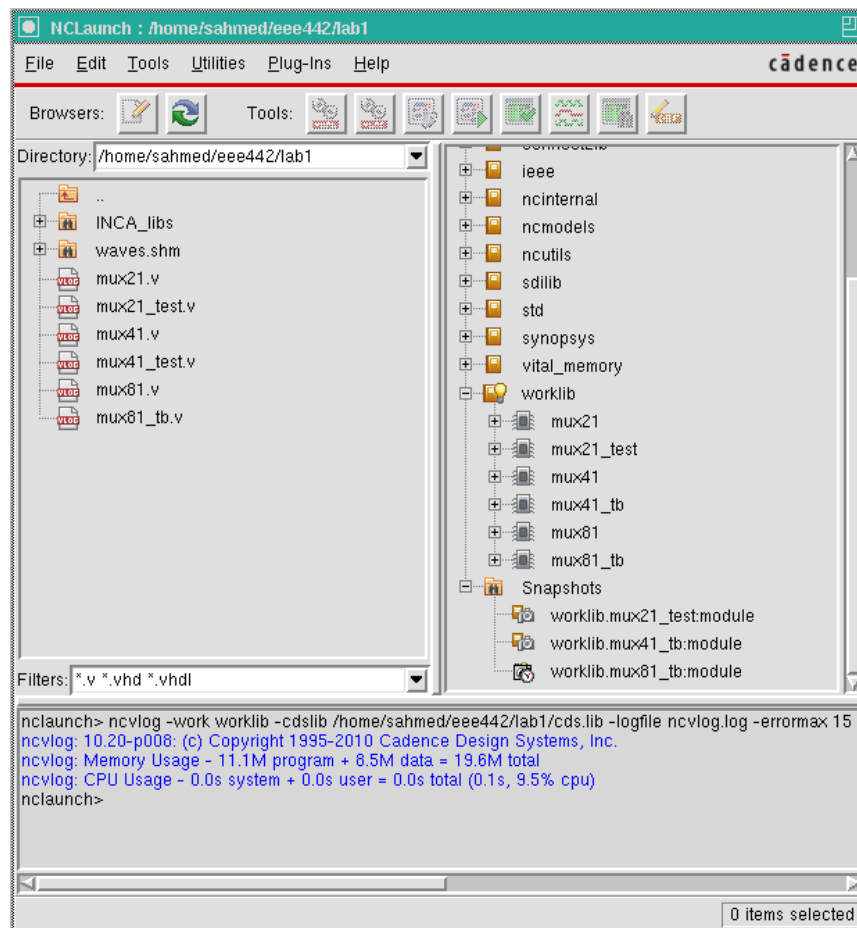


Figure 12: NCLaunch window after mux21, mux41, and mux81 have all been compiled, elaborated, and simulated (worklib and Snapshots populated for all three designs).

Lab Report / Submission Checklist

1. Verilog source code and test bench listings for the 2-to-1, 4-to-1, and 8-to-1 multiplexers.
2. A screenshot of the NCLaunch window showing successful compilation (no errors in the Command Entry Window) for each design.
3. Waveform screenshots from SimVision for each of the three multiplexers, clearly showing the inputs, selector lines, and output.
4. A short discussion (150-200 words) comparing the structural/dataflow description of the 2-to-1 MUX with the behavioral, vector-based description of the 8-to-1 MUX, and explaining how the 4-to-1 MUX was built from 2-to-1 MUX instances.

This lab manual was prepared and formatted by:

Originated: EEE, UIU

Sudipto Mondal

Lecturer, Department of Computer Science and Engineering (CSE)

United International University (UIU)

Email: sudiptomondal@cse.uiu.ac.bd